

Parameter Efficient Fine-Tuning (PEFT) with Batch Processing and Quantization



	TYU division			FRT division		
GHT	254	550	254	274	154	415
RDW	650	320	754	273	825	154
TRG	241	450	144	364	954	174
RTG	254	650	874	657	125	274
WDF	754	155	154	253	744	255
WPT	453	784	054	241	741	245

Contents

01 Introduction of PEFT

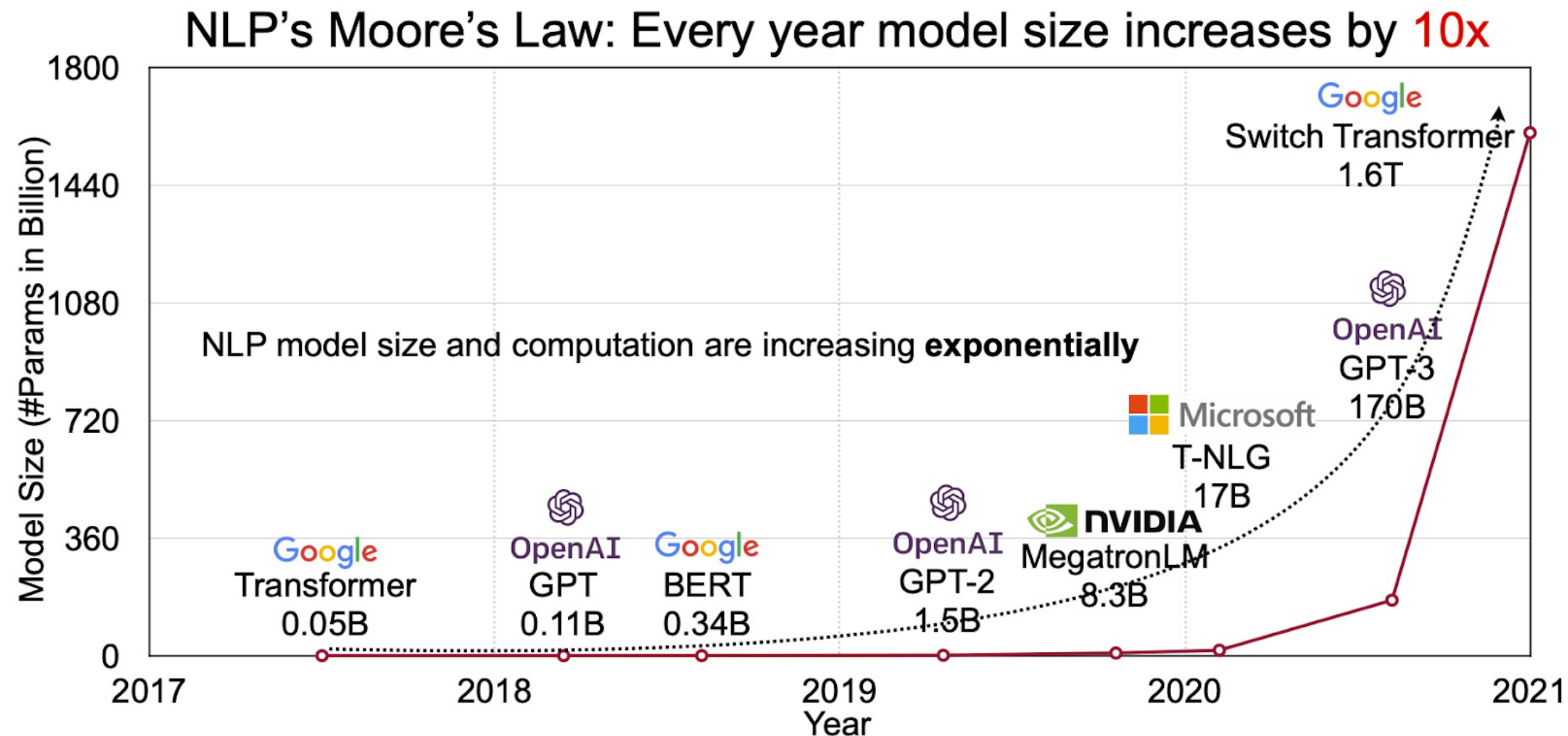
02 LoRA

03 Tips for LoRA



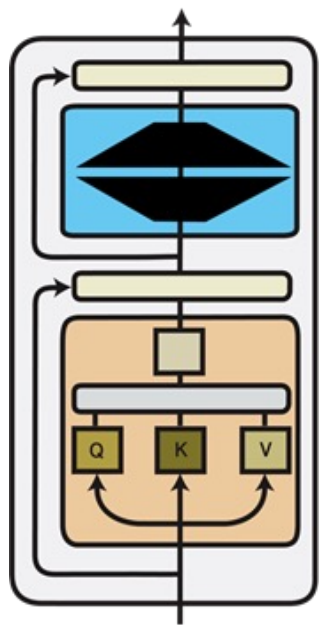
01 Introduction

- LLM models are evolving with a 10x increase in the number of parameters per year
- As parameters increase, the cost of training the model grows exponentially.

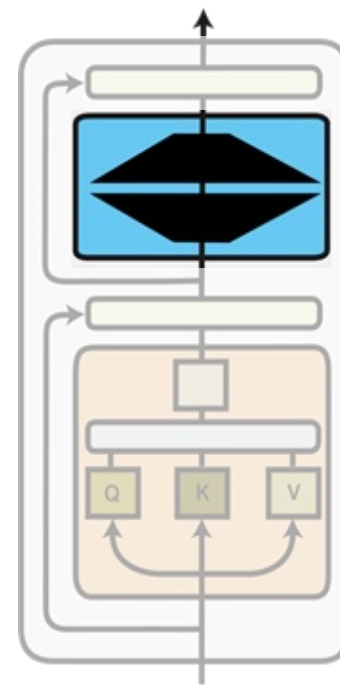


01 Introduction

- From fine-tuning to parameter efficient fine-tuning (PEFT)



Full Fine-tuning
Update **all model parameters**



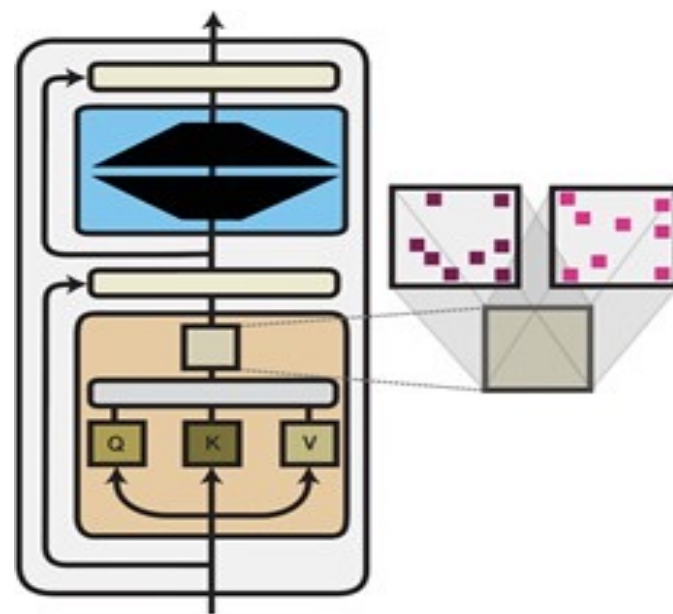
Parameter-efficient Fine-tuning
Update a **small subset of model parameters**

Why fine-tuning *only some* parameters?

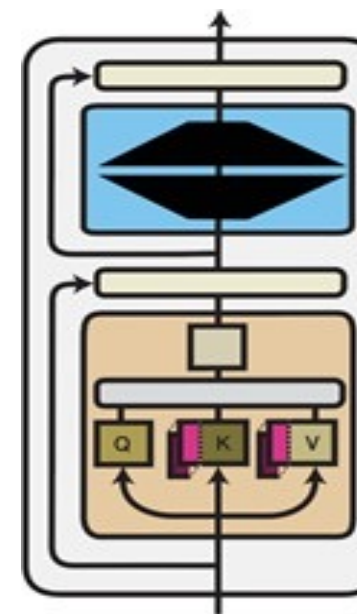
1. Fine-tuning all parameters is impractical with large models
2. State-of-the-art models are massively over-parameterized
→ Parameter-efficient fine-tuning matches performance of full fine-tuning

01 Introduction

- Different perspectives to think about PEFT
 - We can think of three possible aspect of fine-tuning methods using PEFT

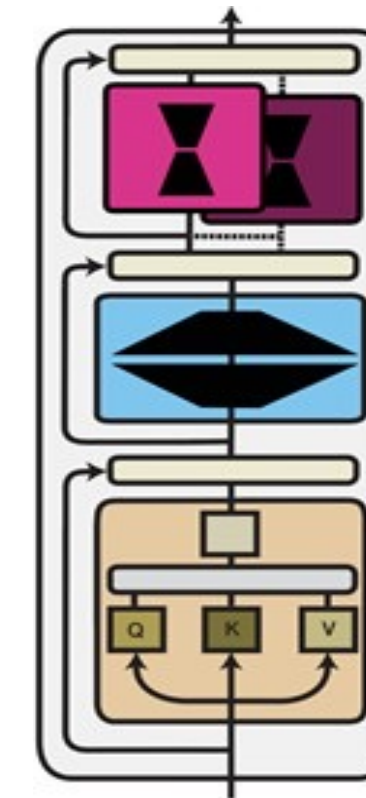


Parameter



Input

Although LoRA appears to modify the model weights, it can also be interpreted as slightly transforming the input representations.



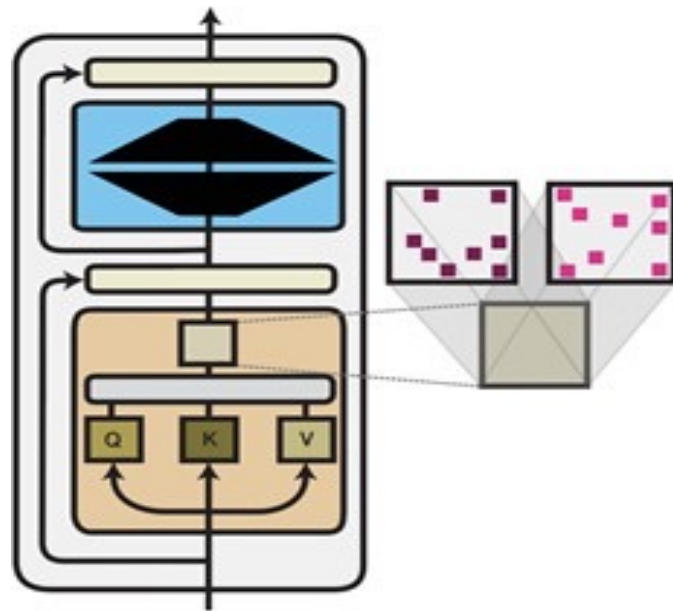
Function

PEFT can ultimately be viewed as a way of slightly modifying the overall function of a model.

$$f'(x) = f(x) + g(x)$$

01 Introduction

- Let just think about the parametric method of PEFT first
 - We want fine-tune our pre-trained model with a small subset of parameters.



Parameter

01 Introduction

- Let just revisit Full-Fine Tuning
Assume we have a pre-trained autoregressive language model $P_\phi(y|x)$ E.g., GPT3
- Adapt this pretrained model to downstream tasks (e.g., summarization, NL2SQL, reading comprehension)
 - Training dataset of context-target pairs $\{(x_i, y_i)\}_{i=1, \dots, N}$
- During full fine-tuning, we update ϕ_o to $\phi_o + \Delta\phi$ by following the gradient to maximize the conditional language modeling objective

$$\max_{\phi} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_{\phi}(y_t | x, y_{<t}))$$

01 Introduction

- For each downstream task, we learn a different set of parameters $\Delta\phi$
 - $|\Delta\phi| = |\phi_o|$
 - GPT-3 has a $|\phi_o|$ of 175 billion
 - Expensive and challenging for storing and deploying many independent instances
- Can we do better?

01 Introduction

- For each downstream task, we learn a different set of parameters $\Delta\phi$
 - $|\Delta\phi| = |\phi_o|$
 - GPT-3 has a $|\phi_o|$ of 175 billion
 - Expensive and challenging for storing and deploying many independent instances
- Can we do better?
- **Key idea:** encode the task-specific parameter increment $\Delta\phi = \Delta\phi(\Theta)$ by a smaller-sized set of parameters Θ , $|\Theta| \ll |\phi_o|$
- The task of finding $\Delta\phi$ becomes optimizing over Θ

$$\max_{\Theta} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_{\phi_o + \Delta\phi(\Theta)}(y_t | x, y_{<t}))$$

Contents

01 Introduction of PEFT

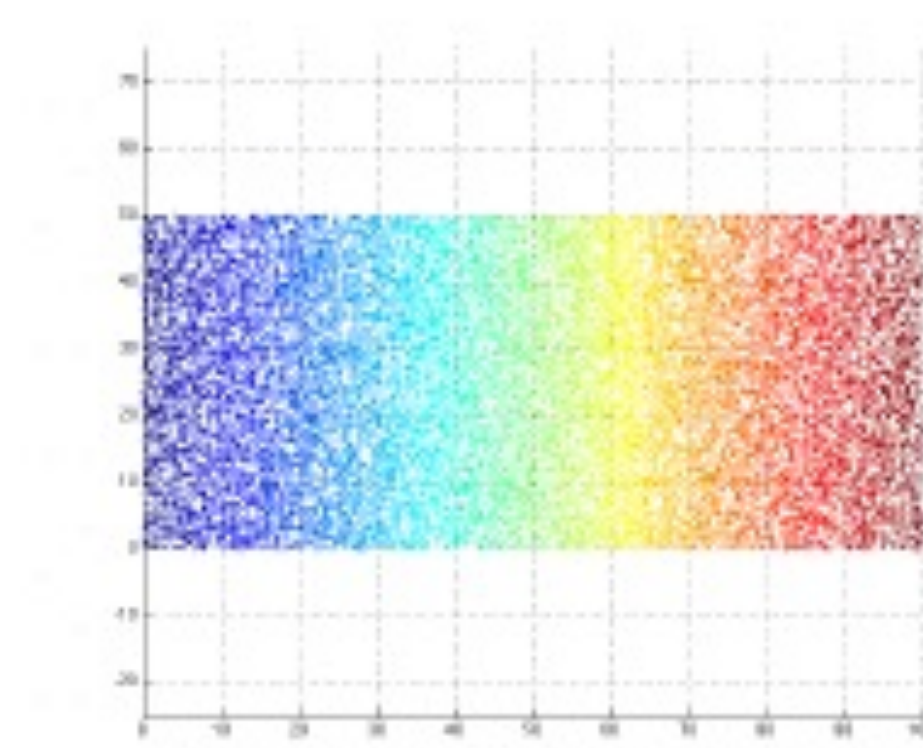
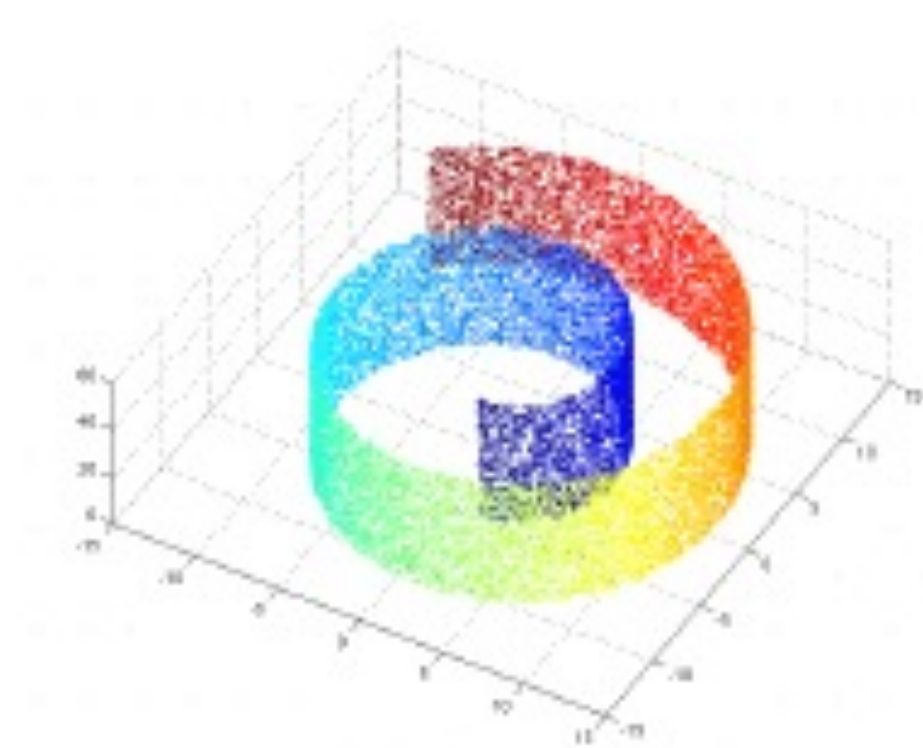
02 LoRA

03 Tips for LoRA



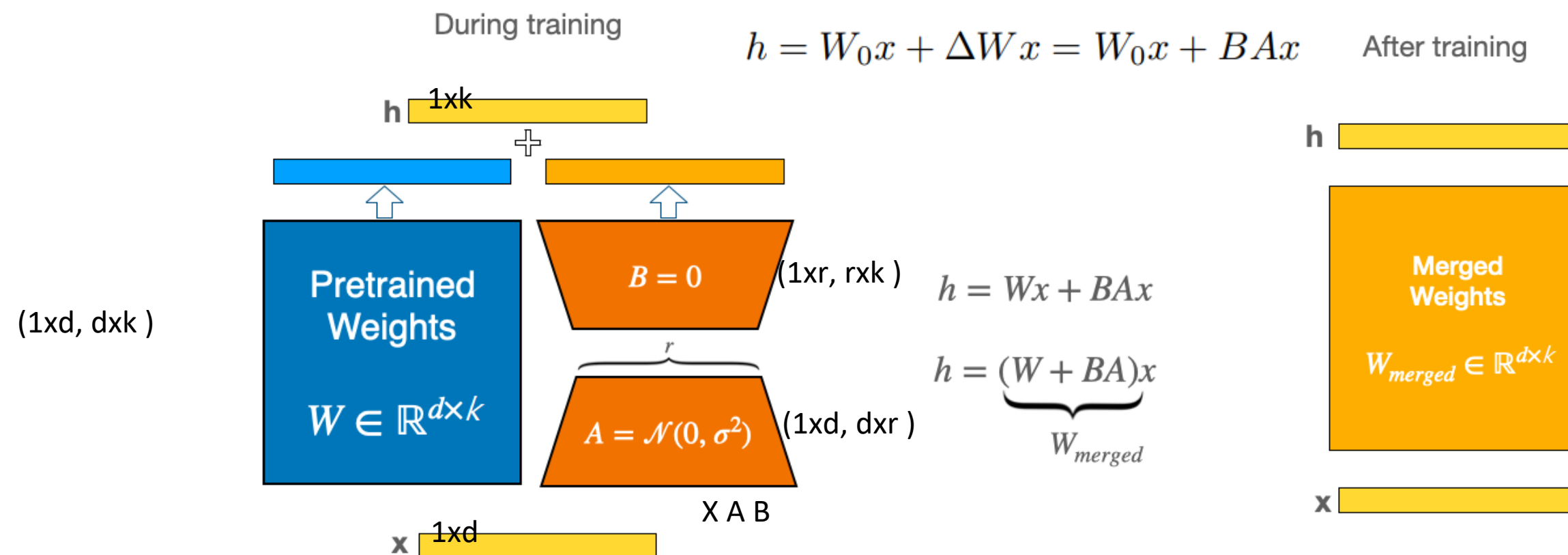
02 LoRA

- LOw-Rank Adaptation (LoRA) claims that a low intrinsic dimension can optionally represent the important features from the massive-parametrized model has a low intrinsic rank.
- This means that regardless of the number of parameters, the important features lies within the low-dimensional topology. : e.g., Dimension Reduction - PCA, LDA, SVD..



02 LoRA

- Normally, to learn the parameters, we compute the gradient of a matrix W of size (d by k).
- Freeze the parameters W of the pre-trained model and train it using a matrix A of size (r by k) and a matrix B of size (d by r).
- Increased training speed and reduced memory usage because it only learns the parameters of LoRA, less than 1% of the total parameters, resulting in increased latency because the matrix product of matrices A and B is summed with the existing parameter W when inferring X



02 LoRA

- Let's think about why it is working, with an example!: We can express a single large matrix as the inner product of two smaller matrices.

$$W = \begin{pmatrix} 6 & 9 & 12 \\ 4 & 6 & 8 \\ 2 & 3 & 4 \end{pmatrix}$$

$$A = \begin{pmatrix} 2 \\ 1.333 \\ 0.666 \end{pmatrix}$$

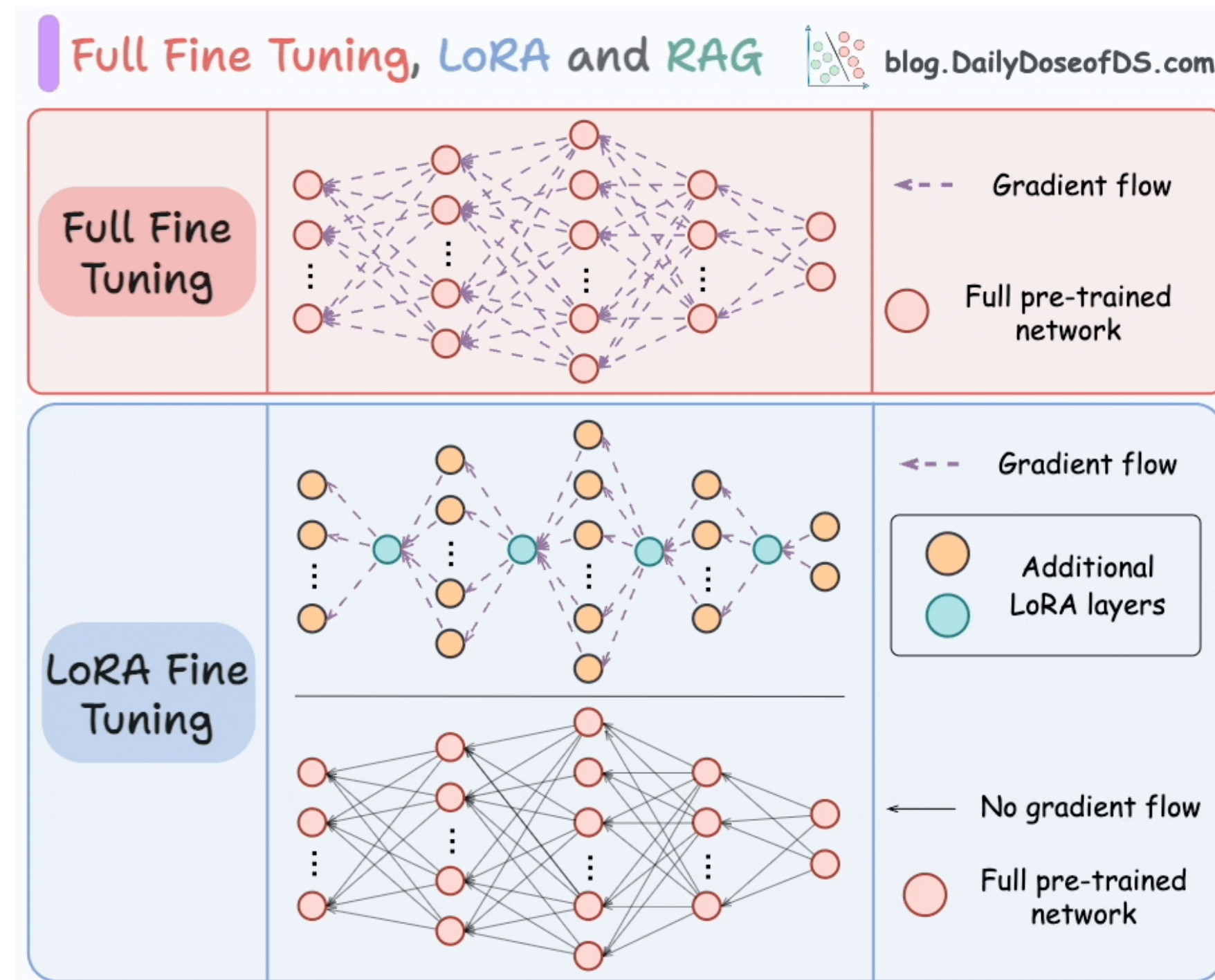
$$B = \begin{pmatrix} 3 & 4.5 & 6 \end{pmatrix}$$

$$\begin{pmatrix} 2 \\ 1.333 \\ 0.666 \end{pmatrix} \begin{pmatrix} 3 & 4.5 & 6 \end{pmatrix} = \begin{pmatrix} 6 & 9 & 12 \\ 4 & 6 & 8 \\ 2.03 & 3.04 & 4.06 \end{pmatrix}$$

$$\text{Gap} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0.03 & 0.04 & 0.06 \end{pmatrix}$$

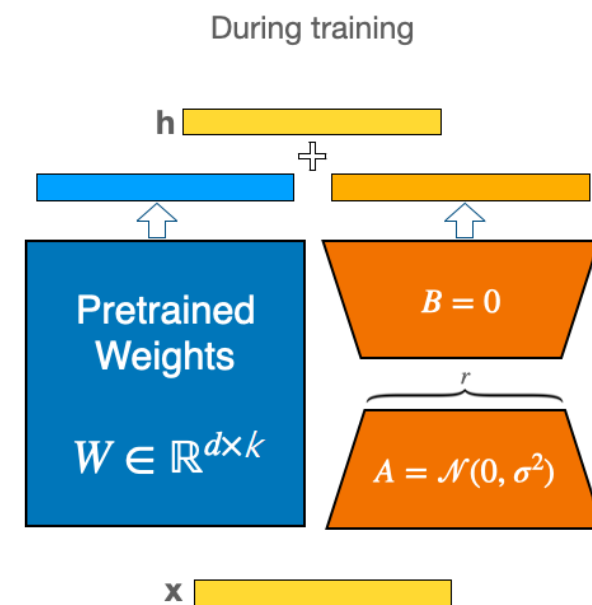
03 LoRA

- Let's think about an example!



03 LoRA

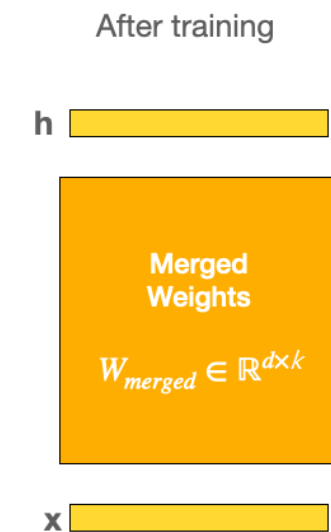
- An Example



```
import torch
import torch.nn as nn
r = 2 # of RANK
d = 3 # of dim of W
k = 3 # of dim of W
W = nn.Linear(d, k)
A = nn.Linear(d, r)
B = nn.Linear(r, k)
```

$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$



```
x = torch.rand(3, d)
model_out = x @ W.weight.T
lora_out = x @ A.weight.T @ B.weight.T
merged_out = model_out + lora_out
```

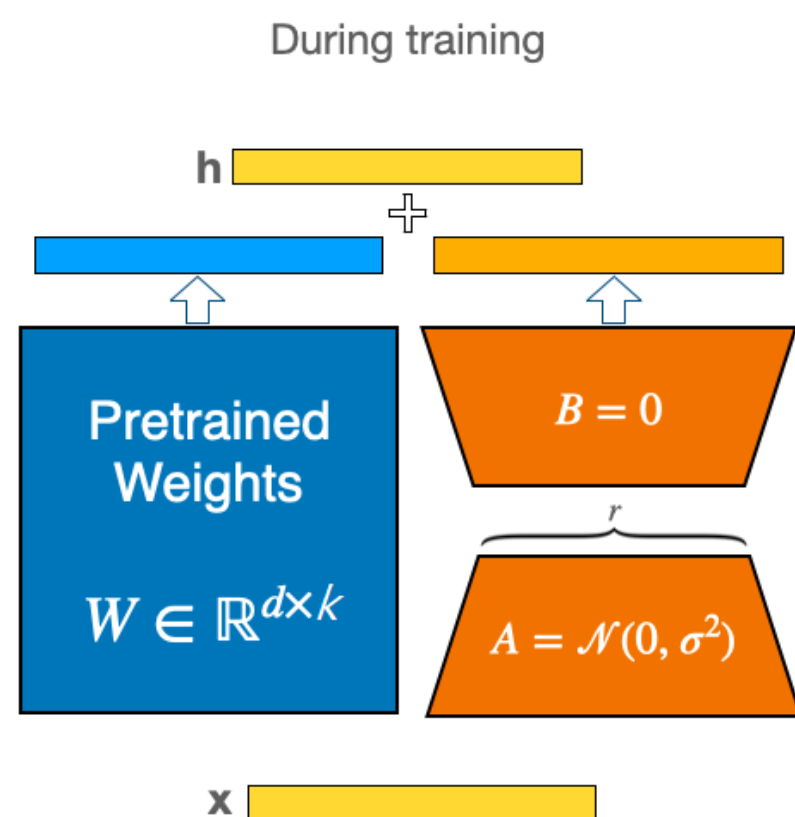
$$h = W_0x + \Delta Wx = W_0x + BAx$$

03 LoRA

- So how is the LoRA Adapter implemented in code?

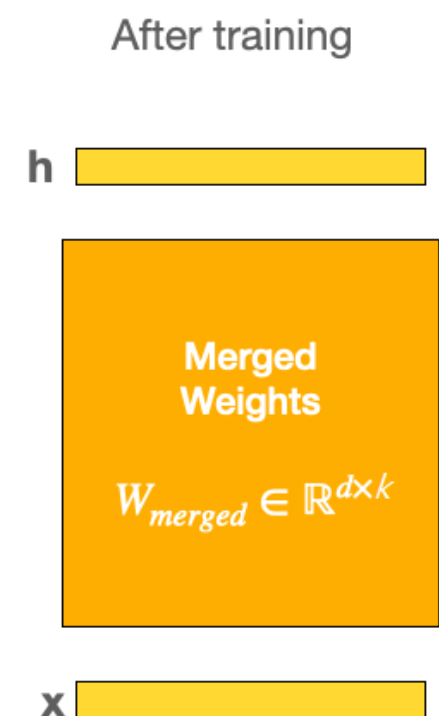


$$h = W_0x + \Delta Wx = W_0x + BAx$$



$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$



03 LoRA

- When you add a LoRA, a LoRALayer is automatically added to the model's layers
 - The example below shows the configuration of a model with and without LoRA in Llama2.

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 5120)
    (layers): ModuleList(
      (0-39): 40 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (k_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (v_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (o_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear4bit(in_features=5120, out_features=13824, bias=False)
          (up_proj): Linear4bit(in_features=5120, out_features=13824, bias=False)
          (down_proj): Linear4bit(in_features=13824, out_features=5120, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm()
        (post_attention_layernorm): LlamaRMSNorm()
      )
    )
    (norm): LlamaRMSNorm()
  )
  (lm_head): Linear(in_features=5120, out_features=32000, bias=False)
)
```

```
PeftModelForCausalLM(
  (base_model): LoraModel(
    (model): LlamaForCausalLM(
      (model): LlamaModel(
        (embed_tokens): Embedding(32000, 5120)
        (layers): ModuleList(
          (0-39): 40 x LlamaDecoderLayer(
            (self_attn): LlamaAttention(
              (q_proj): Linear4bit(
                (lora_dropout): ModuleDict(
                  (default): Dropout(p=0.05, inplace=False)
                )
                (lora_A): ModuleDict(
                  (default): Linear(in_features=5120, out_features=8, bias=False)
                )
                (lora_B): ModuleDict(
                  (default): Linear(in_features=8, out_features=5120, bias=False)
                )
                (lora_embedding_A): ParameterDict()
                (lora_embedding_B): ParameterDict()
                (base_layer): Linear4bit(in_features=5120, out_features=5120, bias=False)
              )
              (k_proj): Linear4bit(
                (lora_dropout): ModuleDict(
                  (default): Dropout(p=0.05, inplace=False)
                )
                ...
              )
            )
            (lm_head): Linear(in_features=5120, out_features=32000, bias=False)
          )
        )
      )
    )
  )
```

trainable params: 358,973,440 || all params: 13,374,837,760 || trainable%: 2.68394612661081

03 LoRA

```
class Linear(nn.Linear, LoRALayer):
    def __init__(...):
        ...
        if r > 0:
            self.lora_A = nn.Parameter(self.weight.new_zeros((r, in_features)))
            self.lora_B = nn.Parameter(self.weight.new_zeros((out_features, r)))
            self.scaling = self.lora_alpha / self.r
            # Freezing the pre-trained weight matrix
            self.weight.requires_grad = False
        ...
```

- When you add a LoRA, a LoRALayer is automatically added to the model's layers

03 LoRA

```
class Linear(nn.Linear, LoRALayer):
    def __init__(...):
        ...
        if r > 0:
            self.lora_A = nn.Parameter(self.weight.new_zeros((r, in_features)))
            self.lora_B = nn.Parameter(self.weight.new_zeros((out_features, r)))
            self.scaling = self.lora_alpha / self.r
            # Freezing the pre-trained weight matrix
            self.weight.requires_grad = False
        ...
```

- When you add a LoRA, a LoRALayer is automatically added to the model's layers

- The “lora_alpha” is a learning rate as a hyperparameter

```
def forward(self, x: torch.Tensor):
    ...
    if self.r > 0 and not self.merged:
        result = F.linear(x, T(self.weight), bias=self.bias)
        if self.r > 0:
            result += (self.lora_dropout(x) @ self.lora_A.T @ self.lora_B.T) * self.scaling
        return result
    ...
```

- Adds the result of passing the frozen layer when $r > 0$ to the matrix product of x, A, B

```
def eval(self):
    def T(w):
        return w.T if self.fan_in_fan_out else w
    nn.Linear.eval(self)
    if self.merge_weights and not self.merged:
        if self.r > 0:
            self.weight.data += T(self.lora_B @ self.lora_A) * self.scaling
        self.merged = True
```

- Adding A, B to W by matrix multiplication in inference (why?)



03 LoRA

```
def forward(self, x: torch.Tensor):  
    ...  
    if self.r > 0 and not self.merged:  
        result = F.linear(x, T(self.weight), bias=self.bias)  
        if self.r > 0:  
            result += (self.lora_dropout(x) @ self.lora_A.T @ self.lora_B.T) * self.scaling  
        return result  
    ...
```

```
def eval(self):  
    def T(w):  
        return w.T if self.fan_in_fan_out else w  
    nn.Linear.eval(self)  
    if self.merge_weights and not self.merged:  
        if self.r > 0:  
            self.weight.data += T(self.lora_B @ self.lora_A) * self.scaling  
        self.merged = True
```

- When you add a LoRA, a LoRALayer is automatically added to the model's layers
- Adding A,B to W by matrix multiplication in inference (why?)
--> This is because the calculation results are the same!
1) The two matrices A,B are pre-multiplied and then X is computed.
2) W_0 is added to the BA matrix and then X is computed.

03 LoRA

- Adding A,B to W by matrix multiplication in inference (why?)

--> This is because the calculation results are the same!

1) The two matrices A,B are pre-multiplied and then X is computed.

2) W_0 is added to the BA matrix and then X is computed.

```
1 import torch
2 import torch.nn as nn
3 r = 2 # of RANK
4 d = 5 # of dim of W
5 k = 7 # of dim of W
6 W = nn.Linear(d,k)
7 A = nn.Linear(d,r)
8 B = nn.Linear(r,k)
```

✓ 0.0s

```
1 x = torch.rand(3,d)
```

✓ 0.0s

```
1 model_out = x @ W.weight.T
2 print(model_out)
```

✓ 0.0s

```
tensor([[ 0.2623,  0.5047, -0.3954,  0.0075, -0.2403,  0.7900, -0.1682],
        [ 0.1031,  0.3503, -0.5271, -0.0959, -0.4584,  0.5662,  0.3333],
        [ 0.1906,  0.3685, -0.1884, -0.0803, -0.2886,  0.5590,  0.1233]],
        grad_fn=<MmBackward0>)
```

```
1 lora_out = x @ A.weight.T @ B.weight.T
2 print(lora_out)
```

✓ 0.0s

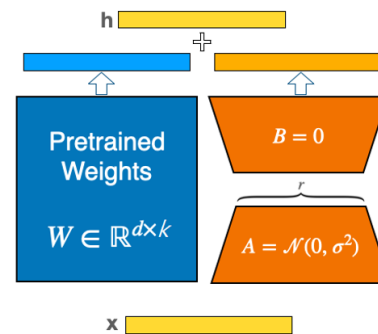
```
tensor([[ -0.3777, -0.0277,  0.1669,  0.2973,  0.0817,  0.0033,  0.2428],
        [-0.1907, -0.0613,  0.0772,  0.1583,  0.1567, -0.1064,  0.2359],
        [-0.2603, -0.0422,  0.1115,  0.2089,  0.1126, -0.0505,  0.2227]],
        grad_fn=<MmBackward0>)
```

+ Code

```
1 merged_out = model_out + lora_out
2 print(merged_out)
```

✓ 0.0s

```
tensor([[ -0.1154,  0.4771, -0.2285,  0.3047, -0.1586,  0.7934,  0.0746],
        [-0.0876,  0.2890, -0.4500,  0.0624, -0.3017,  0.4598,  0.5692],
        [-0.0698,  0.3263, -0.0768,  0.1286, -0.1760,  0.5085,  0.3459]],
        grad_fn=<AddBackward0>)
```



```
1 tmp = A.weight.T @ B.weight.T
2 print(tmp.T)
```

✓ 0.0s

```
tensor([[ -0.2936,  0.0376,  0.2415,  0.1734, -0.3395],
        [-0.0212,  0.0711,  0.0258, -0.0193, -0.0683],
        [ 0.1298, -0.0064, -0.1055, -0.0814,  0.1435],
        [ 0.2310, -0.0414, -0.1915, -0.1310,  0.2747],
        [ 0.0626, -0.1747, -0.0721,  0.0406,  0.1792],
        [ 0.0034,  0.1557,  0.0164, -0.0747, -0.0961],
        [ 0.1879, -0.1877, -0.1747, -0.0348,  0.3221]],
        grad_fn=<PermuteBackward0>)
```

```
1 W.weight
```

✓ 0.0s

Parameter containing:

```
tensor([[ 0.2063,  0.1595, -0.1073, -0.0308,  0.0739],
        [ 0.4053,  0.1043,  0.2600, -0.1446, -0.0242],
        [ 0.0635,  0.0274, -0.0766, -0.1748, -0.4459],
        [ 0.0420,  0.1410,  0.3899, -0.2170, -0.3356],
        [-0.0914, -0.0611, -0.3559, -0.3673,  0.1161],
        [ 0.4332,  0.3423, -0.0943,  0.1022,  0.3160],
        [-0.0883, -0.4124,  0.2665,  0.4421, -0.0681]], requires_grad=T)
```

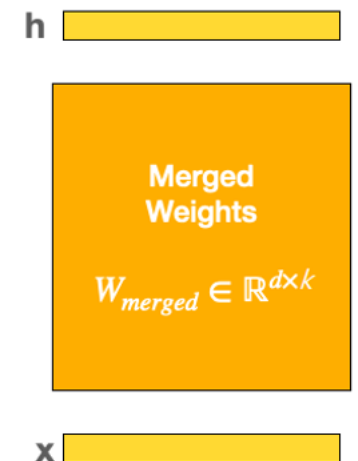
+ Code

```
1 x@(W.weight+tmp.T).T
```

✓ 0.0s

```
tensor([[ -0.1154,  0.4771, -0.2285,  0.3047, -0.1586,  0.7934,  0.0746],
        [-0.0876,  0.2890, -0.4500,  0.0624, -0.3017,  0.4598,  0.5692],
        [-0.0698,  0.3263, -0.0768,  0.1286, -0.1760,  0.5085,  0.3459]],
        grad_fn=<MmBackward0>)
```

After training



03 LoRA

- Number of parameters in KoBERT

```
print_trainable_parameters(bertmodel)
```

```
trainable params: 92186880 || all params: 92186880 || trainable%: 100.00
```

- KoBERT with LoRA and the number of parameters in the final classification model

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=4,
    lora_alpha=32,
    target_modules=["query", "value"],
    lora_dropout=0.1,
    bias="lora_only",
    modules_to_save=["decode_head"],
)
lora_model = get_peft_model(bertmodel, config)
print_trainable_parameters(lora_model)
```

```
trainable params: 165888 || all params: 92334336 || trainable%: 0.18
```

```
print_trainable_parameters(model)
```

```
trainable params: 167426 || all params: 92335874 || trainable%: 0.18
```


03 LoRA

- Modify classifier's weights to be able to save and retrieve them as LoRA's weights support saving and retrieving from the PEFT library.

```
class BERTClassifier(nn.Module):
    def __init__(self, bert, hidden_size=768, num_classes=2, dr_rate=None, params=None):
        ...

    def gen_attention_mask(self, token_ids, valid_length):
        ...

    def forward(self, token_ids, valid_length, segment_ids):
        ...

    def save_classifier(self, file_path):
        torch.save(self.classifier.state_dict(), file_path)

    def load_classifier(self, file_path):
        self.classifier.load_state_dict(torch.load(file_path))
```

```
lora_model.save_pretrained('kobert_nsmc_classifier_weights')

from peft import PeftModel
inference_model = PeftModel.from_pretrained(loadbertmodel, "kobert_nsmc_classifier_weights")
```

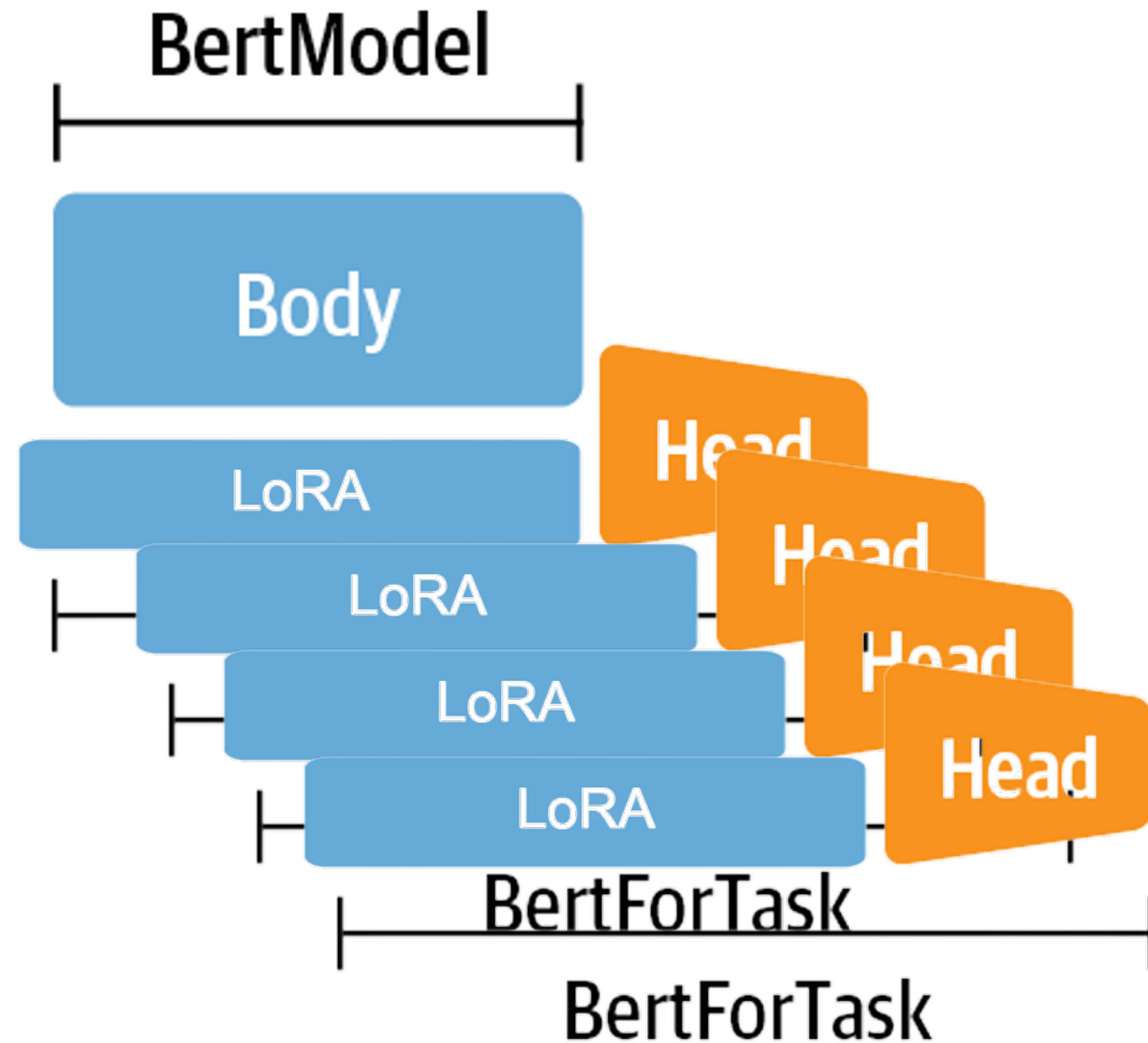
03 LoRA

- The comparison between the existing model, the model that only freezes the parameters of KoBERT, and the model using LoRA is as follows.

	KoBERT+Classifier	Classifier	LoRA+Classifier
Duration for 1epoch	13:31	4:42	9:51
Accuracy	89.83%	66.83%	85.46%
# of parameters	92,335,874(100%)	1,538(0.00%)	167,426(0.18%)

- We can see that the model using LoRA has a shorter learning time and slightly lower accuracy than the traditional model, and a significantly higher accuracy than the model with KoBERT's parameter frozen.
- With LoRA, the amount of parameters to be learned is greatly reduced, but it is assumed that the learning time does not decrease by the amount of reduced parameters due to dataloaders, etc.

03 LoRA



- With LoRA, you can save resources by loading the base model and changing the LoRA trained on the task from task to task, rather than having to load a model that has been fine-tuned to the task every time the task changes.

03 LoRA

- The result of applying LoRA to multiple layers while limiting the amount of parameters that can be trained is as follows

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

- Performed best when applied to Q, V or Q, K, V, O

03 LoRA

- Depending on the applied layer and dimension r , the results are as follows

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

- We can see that performance does not necessarily increase as the dimensionality r of LoRA increases, which supports our earlier claim that low intrinsic rank exists in learning.

Contents

01 Introduction

02 LoRA

03 Tips for LoRA



Tips for LoRA

- Reference

[← BACK TO BLOG](#)

Finetuning LLMs with LoRA and QLoRA: Insights from Hundreds of Experiments

Posted on October 12, 2023 by [Sebastian Raschka](#) - [Blog](#), [Community](#), [Tutorials](#)

Tips for LoRA

- Different models have different structures, so you need to check the model once

KT-AI/midm-bitext-S-7B-inst-v1

```
MidmLMHeadModel(
  (transformer): MidmModel(
    (wte): Embedding(72192, 4096)
    (rotary_pos_emb): RotaryEmbedding()
    (drop): Dropout(p=0.0, inplace=False)
    (h): ModuleList(
      (0-31): 32 x MidmBlock(
        (ln_1): LayerNorm((4096,), eps=1e-05, elementwise_affine=True)
        (attn): MidmAttention(
          (c_attn): Linear(in_features=4096, out_features=12288, bias=True)
          (c_proj): Linear(in_features=4096, out_features=4096, bias=True)
          (attn_dropout): Dropout(p=0.0, inplace=False)
          (resid_dropout): Dropout(p=0.0, inplace=False)
        )
        (ln_2): LayerNorm((4096,), eps=1e-05, elementwise_affine=True)
        (mlp): MidmMLP(
          (c_fc): Linear(in_features=4096, out_features=21760, bias=True)
          (c_proj): Linear(in_features=10880, out_features=4096, bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((4096,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=4096, out_features=72192, bias=False)
)
```

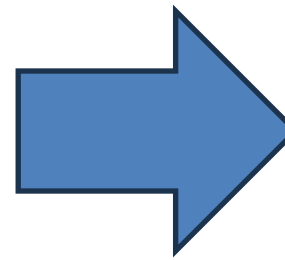
beomi/llama-2-ko-7b

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(46336, 4096, padding_idx=0)
    (layers): ModuleList(
      (0-31): 32 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=4096, out_features=4096, bias=False)
          (k_proj): Linear(in_features=4096, out_features=4096, bias=False)
          (v_proj): Linear(in_features=4096, out_features=4096, bias=False)
          (o_proj): Linear(in_features=4096, out_features=4096, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=4096, out_features=11008, bias=False)
          (up_proj): Linear(in_features=4096, out_features=11008, bias=False)
          (down_proj): Linear(in_features=11008, out_features=4096, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm()
        (post_attention_layernorm): LlamaRMSNorm()
      )
    )
    (norm): LlamaRMSNorm()
  )
  (lm_head): Linear(in_features=4096, out_features=46336, bias=False)
)
```

Tips for LoRA

- Different models have different structures, so you need to check the model once

```
LlamaForCausalLM(  
  (model): LlamaModel(  
    (embed_tokens): Embedding(46336, 4096, padding_idx=0)  
    (layers): ModuleList(  
      (0-31): 32 x LlamaDecoderLayer(  
        (self_attn): LlamaAttention(  
          (q_proj): Linear(in_features=4096, out_features=4096, bias=False)  
          (k_proj): Linear(in_features=4096, out_features=4096, bias=False)  
          (v_proj): Linear(in_features=4096, out_features=4096, bias=False)  
          (o_proj): Linear(in_features=4096, out_features=4096, bias=False)  
          (rotary_emb): LlamaRotaryEmbedding()  
        )  
        (mlp): LlamaMLP(  
          (gate_proj): Linear(in_features=4096, out_features=11008, bias=False)  
          (up_proj): Linear(in_features=4096, out_features=11008, bias=False)  
          (down_proj): Linear(in_features=11008, out_features=4096, bias=False)  
          (act_fn): SiLUActivation()  
        )  
        (input_layernorm): LlamaRMSNorm()  
        (post_attention_layernorm): LlamaRMSNorm()  
      )  
    )  
    (norm): LlamaRMSNorm()  
  )  
  (lm_head): Linear(in_features=4096, out_features=46336, bias=False)  
)
```

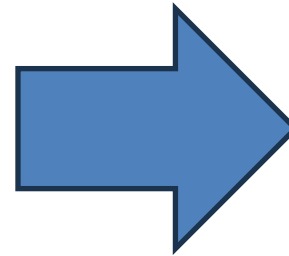


```
peft_config = LoraConfig(  
  target_modules=["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj", "down_proj", "up_proj"],  
  inference_mode=False,  
  r=32,  
  lora_alpha=64,  
  lora_dropout=0.05,  
  modules_to_save=["embed_tokens", "lm_head"]  
)
```

Tips for LoRA

- Different models have different structures, so you need to check the model once

```
MidmLMHeadModel(
  (transformer): MidmModel(
    (wte): Embedding(72192, 4096)
    (rotary_pos_emb): RotaryEmbedding()
    (drop): Dropout(p=0.0, inplace=False)
    (h): ModuleList(
      (0-31): 32 x MidmBlock(
        (ln_1): LayerNorm((4096,), eps=1e-05, elementwise_affine=True)
        (attn): MidmAttention(
          (c_attn): Linear(in_features=4096, out_features=12288, bias=True)
          (c_proj): Linear(in_features=4096, out_features=4096, bias=True)
          (attn_dropout): Dropout(p=0.0, inplace=False)
          (resid_dropout): Dropout(p=0.0, inplace=False)
        )
        (ln_2): LayerNorm((4096,), eps=1e-05, elementwise_affine=True)
        (mlp): MidmMLP(
          (c_fc): Linear(in_features=4096, out_features=21760, bias=True)
          (c_proj): Linear(in_features=10880, out_features=4096, bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
    )
    (ln_f): LayerNorm((4096,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=4096, out_features=72192, bias=False)
)
```



```
peft_config = LoraConfig(
  target_modules=["c_attn", "c_proj", "c_fc"],
  inference_mode=False,
  r=32,
  lora_alpha=64,
  lora_dropout=0.05,
  modules_to_save=["wte", "lm_head"]
)
```


Tips for LoRA

- Write the LoraConfig and you're done!

```
peft_config = LoraConfig(
    target_modules=["c_attn", "c_proj", "c_fc"],
    inference_mode=False,
    r=32,
    lora_alpha=64,
    lora_dropout=0.05,
    modules_to_save=["wte", "lm_head"]
)
```

```
model = AutoModelForCausalLM.from_pretrained(
    "KT-AI/midm-bitext-S-7B-inst-v1",
    trust_remote_code=True
)
model = get_peft_model(model, peft_config)
```

✓ 1m 4.4s

model

✓ 0.0s

```
PeftModel(
  (base_model): LoraModel(
    (model): MidmLMHeadModel(
      (transformer): MidmModel(
        (wte): ModulesToSaveWrapper(
          (original_module): Embedding(72192, 4096)
          (modules_to_save): ModuleDict(
            (default): Embedding(72192, 4096)
          )
        )
        (rotary_pos_emb): RotaryEmbedding()
        (drop): Dropout(p=0.0, inplace=False)
        (h): ModuleList(
          (0-31): 32 x MidmBlock(
            (ln_1): LayerNorm((4096,)), eps=1e-05, elementwise_affine=True)
            (attn): MidmAttention(
              (c_attn): Linear(
                in_features=4096, out_features=12288, bias=False
                (lora_dropout): ModuleDict(
                  (default): Dropout(p=0.05, inplace=False)
                )
                (lora_A): ModuleDict(
                  (default): Linear(in_features=4096, out_features=32, bias=False)
                )
                (lora_B): ModuleDict(
```

...

Tips for LoRA

❖ Caution

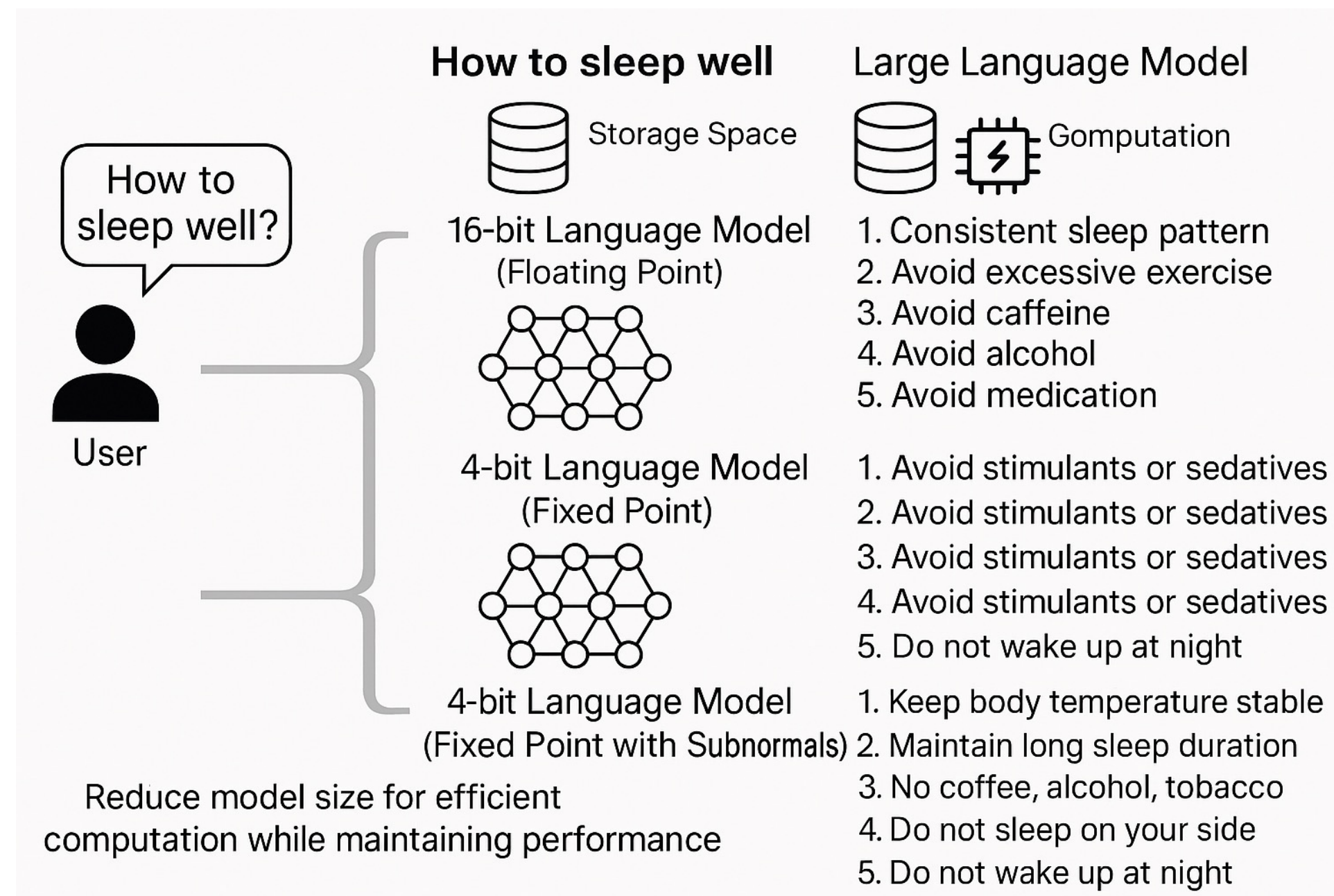
- When using the Peft library to import a base model and a trained lora, be aware that the original base model and the combined lora will be the same model.

```
from peft import PeftModel  
  
model_with_lora = PeftModel.from_pretrained(model, lora)
```

- This means that running the above code will result in `model_with_lora = model`
- Therefore, the base model must be reloaded to compare with the base model

02 QLoRA

- The quantization methods based on LoRA are proposed for better efficiency
- But when we apply quantization methods, training model is impossible since the 4bit is not enough



02 QLoRA

- What is Quantization?

32-bit floating point

Byte 1

Byte 2

Byte 3

Byte 4

1 Billion Parameters x 4 Bytes → 4 Billion Bytes

1 GB → 1,073,741,824 Bytes

4 Billion Bytes / 1,073,741,824 Bytes → 3.73 GB

02 QLoRA

- What is Quantization?

32-bit floating point

Byte 1

Byte 2

Byte 3

Byte 4



16-bit floating point

Byte 1

Byte 2

Or

4-bits

4-bits

8-bit int

Byte 1

2-bits

2-bits

02 QLoRA

- What is Quantization?

Precision Type	Memory Usage (vs FP32)	Performance Trade-offs
FP32 (Full Precision)	100% (default)	Best accuracy, high compute cost
FP16 (Half Precision)	50%	Good balance of speed and accuracy
INT8 (8-bit Integer)	25%	Minor accuracy loss, big speed gain
INT4 (4-bit Integer)	12.5%	Noticeable accuracy loss, very fast
INT2 (2-bit Integer)	6.25%	Extreme compression, high accuracy loss

02 QLoRA

- What is Quantization?

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 5120)
    (layers): ModuleList(
      (0-39): 40 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (k_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (v_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (o_proj): Linear4bit(in_features=5120, out_features=5120, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear4bit(in_features=5120, out_features=13824, bias=False)
          (up_proj): Linear4bit(in_features=5120, out_features=13824, bias=False)
          (down_proj): Linear4bit(in_features=13824, out_features=5120, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm()
        (post_attention_layernorm): LlamaRMSNorm()
      )
    )
    (norm): LlamaRMSNorm()
  )
  (lm_head): Linear(in_features=5120, out_features=32000, bias=False)
)
```

```
PeftModelForCausalLM(
  (base_model): LoraModel(
    (model): LlamaForCausalLM(
      (model): LlamaModel(
        (embed_tokens): Embedding(32000, 5120)
        (layers): ModuleList(
          (0-39): 40 x LlamaDecoderLayer(
            (self_attn): LlamaAttention(
              (q_proj): Linear4bit(
                (lora_dropout): ModuleDict(
                  (default): Dropout(p=0.05, inplace=False)
                )
                (lora_A): ModuleDict(
                  (default): Linear(in_features=5120, out_features=8, bias=False)
                )
                (lora_B): ModuleDict(
                  (default): Linear(in_features=8, out_features=5120, bias=False)
                )
                (lora_embedding_A): ParameterDict()
                (lora_embedding_B): ParameterDict()
                (base_layer): Linear4bit(in_features=5120, out_features=5120, bias=False)
              )
            (k_proj): Linear4bit(
              (lora_dropout): ModuleDict(
                (default): Dropout(p=0.05, inplace=False)
              )
            )
          )
        )
        ...
        (lm_head): Linear(in_features=5120, out_features=32000, bias=False)
      )
    )
  )
)
```

trainable params: 358,973,440 || all params: 13,374,837,760 || trainable%: 2.68394612661081

02 QLoRA

- QLoRA doesn't accelerate your learning
- However, QLoRA does save your VRAM usage.

Default LoRA (with bfloat-16):

- Training time: 6685.75s
- Memory used: 21.33 GB

QLoRA via `--quantize "bnb.nf4"`:

- Training time: 10059.53s
- Memory used: 14.18 GB

QLoRA via `-quantize "bnb.fp4"`:

- Training time: 9334.45s
- Memory used: 14.19 GB

	TruthfulQA MC1	TruthfulQA MC2	Arithmetic 2ds	Arithmetic 4ds	BLiMP Causative	MMLU Global Facts
Llama 2 7B base	0.2534	0.3967	0.508	0.637	0.787	0.32
LoRA default 1	0.2876	0.4211	0.3555	0.0035	0.75	0.27
LoRA default 2	0.284	0.4217	0.369	0.004	0.747	0.26
LoRA default 3	0.2815	0.4206	0.372	0.004	0.747	0.27
QLoRA (nf4)	0.2803	0.4139	0.4225	0.006	0.783	0.23
QLoRA (fp4)	0.2742	0.4047	0.5295	0.016	0.744	0.23

- No significant difference in performance
- QLoRA is the best!

03 etc.

- AdamW is more VRAM hungry than SGD because it uses 2 parameters for every 1 model parameter, but 14.15 GB (SGD) -> 14.18 GB (AdamW) is not significantly different
- AdamW had a learning rate of $3e-4$ and a decay rate of 0.01, while SGD had a learning rate of 0.1 and a momentum of 0.9.
- For scheduler, we recommend the cosine scheduler
- For the alpaca dataset (50k), will 2 runs have the same effect as 100k runs? -> Worse
- Doubling r does not double VRAM $8 \rightarrow 16 \Rightarrow 16.42 \text{ GB} \rightarrow 16.47 \text{ GB}$
- r scales well from 8 -> 16 -> 32 -> ... -> 256, with negligible performance increase beyond that.
- Target module applies all applicable linear modules
- α is twice r is good in any situation, memorize it if you don't know it $\alpha = r \times 2$

Contents

01 Introduction of PEFT

02 LoRA

03 Tips for LoRA

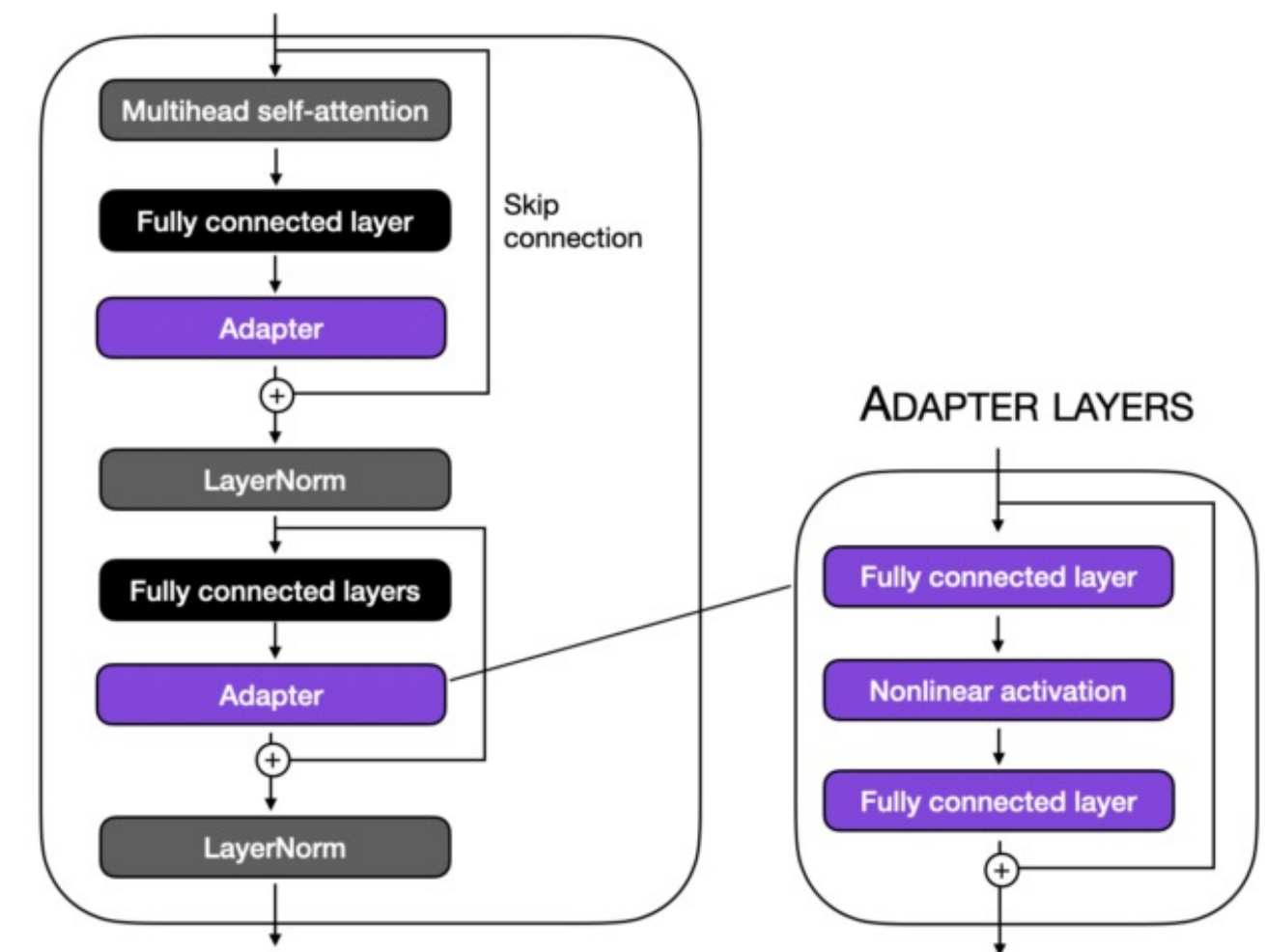
04 Other PEFT methods



Adapter-based models

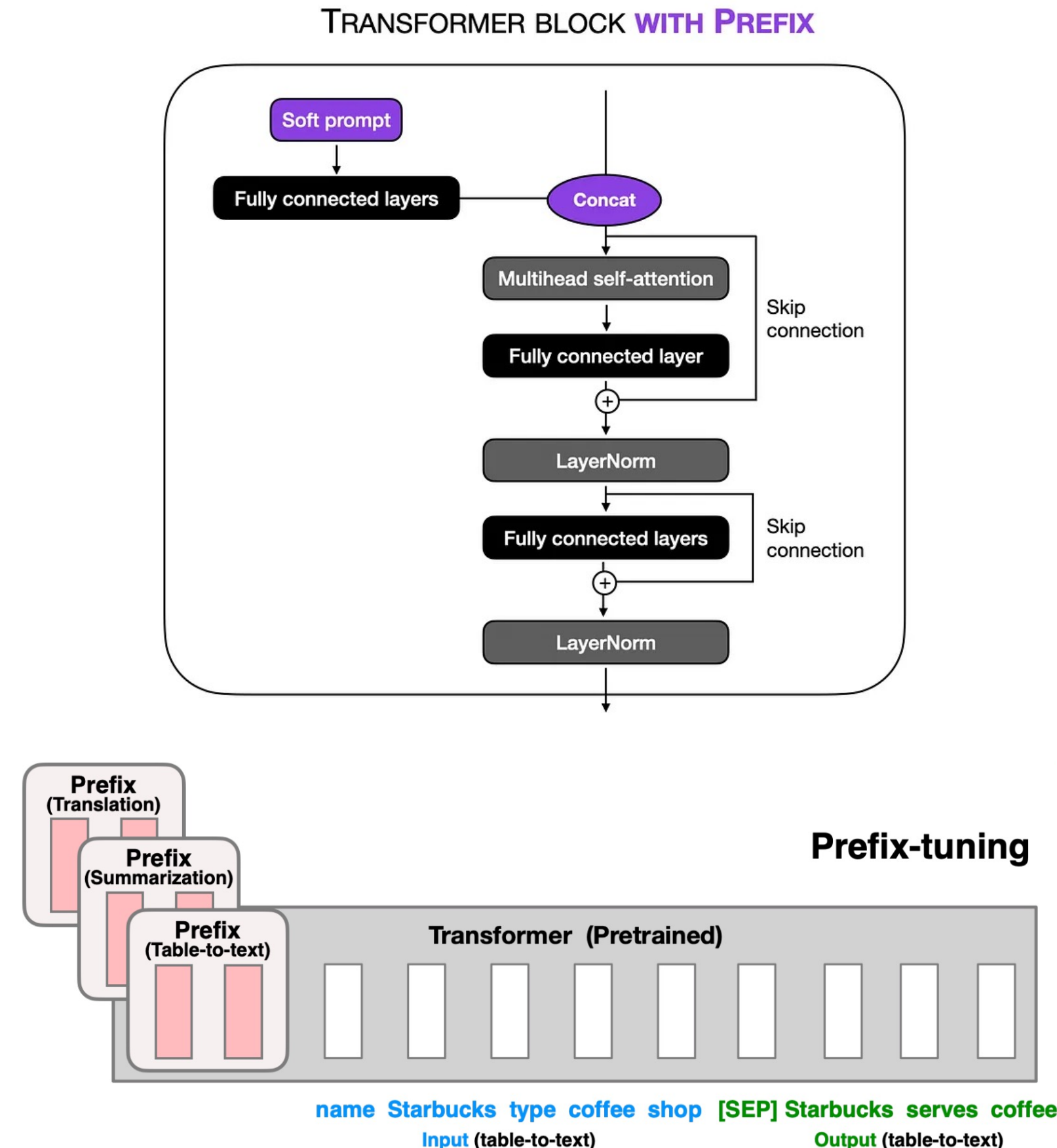
- Previous research also suggests that learning the full parameters of a model is inefficient.
- Several adapters proposed, but the most basic one is How to add an adapter to a **Linear Layer** result.
- Utilizes a method that **freezes** the parameters of the **pre-trained model** and **only trains adapter parameters** that are less than 1% of the total parameters.

TRANSFORMER BLOCK **WITH ADAPTERS**



Prefix tuning models

- Add a prefix tokens that provides additional information about the input sequence to feed the input sequence and the added information into the Attention layer.
- The amount of parameters trained is very small, making it very difficult to improve performance.
- Added information causes the maximum sequence length of the model to become smaller



Thank you